Generalization-First Object Detection: Winning the NorgesGruppen Shelf Recognition Challenge

NM i AI 2026 — 1st Place Solution (0.7113 mAP)

Tom Daniel Grande Henrik Skulevold Tobias Korten Fridtjof Hoyer

Team Experis

NM i AI 2026 Competition, March 19–22, 2026

Abstract

Congratulations to all participants and winners of NM i AI 2026, and a special thanks to the organizers at Astar Technologies, the task sponsors NorgesGruppen Data, Tripletex, and Astar, and all volunteers who made the competition possible.

We present our solution to the NorgesGruppen grocery shelf product detection task (Task 3), which achieved 0.7113 mAP on the private test set. The task requires localizing and classifying 356 fine-grained grocery product categories on retail shelf images, scored as $0.7 \times \text{mAP}_{\text{det}} + 0.3 \times \text{mAP}_{\text{cls}}$. Our key insight was that generalization to the unseen private test distribution — not training-set accuracy — determines the final ranking. We built a 3-model ensemble combining three distinct YOLO architectures (YOLOv8x, YOLO11x, YOLOv8l) with diverse augmentation strategies, fused via multi-scale inference and Weighted Boxes Fusion. Over 69 hours we executed 150+ training runs on Azure ML, systematically eliminating approaches that improved local metrics but failed to generalize.

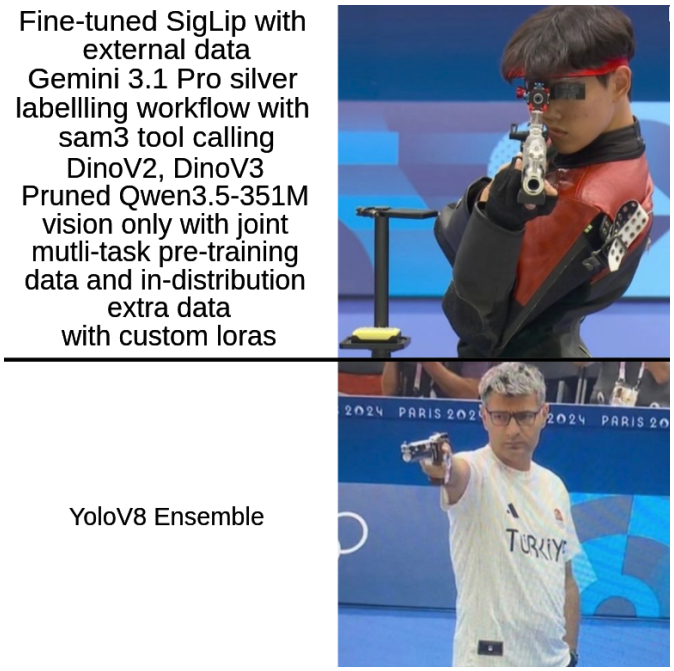

Figure 1: The competition experience, summarized. Many teams built sophisticated pipelines combining fine-tuned vision-language models, generative AI for synthetic data, and contrastive learning — only to be outperformed by a well-configured YOLO ensemble trained on the provided data alone. Meme credit: Markus Hei (NM i AI 2026 Slack).

1 Introduction

NM i AI is Norway's national AI competition, held as a 69-hour event where teams compete across three independent challenges. The 2026 edition ran March 19–22 with tasks in AI agents (Tripletex), prediction engines (Astar), and object detection (NorgesGruppen Data). Teams' overall ranking is determined by normalized scores across all three tasks, though each task also has its own leaderboard.

Task 3 asked participants to build a product detection system for grocery store shelves. Given a photograph of a store shelf section, the system must detect every visible product with a bounding box and classify it into one of 356 product categories. The score combines detection and classification quality:

$$\text{Score} = 0.7 \times \text{mAP}_{\text{det}}@0.5 + 0.3 \times \text{mAP}_{\text{cls}}@0.5 \tag{1}$$

Submissions ran on a sandboxed server with Python 3.11, PyTorch 2.6.0, Ultralytics 8.1.0, an NVIDIA L4 GPU, a 300-second time limit, no network access, and a 420 MB ZIP size cap. Teams could submit at most three times per day. Critically, the public leaderboard used during the competition was scored on a different test set from the final private ranking — making generalization the decisive factor.

2 The Domain: Grocery Shelf Recognition

Automated shelf recognition is a growing area of applied computer vision, driven by the retail industry's need for real-time inventory monitoring, planogram compliance checking, and out-of-stock detection. NorgesGruppen, Norway's largest grocery retailer, provided the dataset for this challenge. The most closely related public benchmark is SKU-110K [Goldman et al.(2019)], a densely packed shelf detection dataset with 11,762 images. Our task is harder in one key dimension (356 fine-grained classes vs. single-class detection) but easier in others (less dense, canonical orientations).

The domain has characteristics that distinguish it from standard object detection benchmarks. Products are densely packed in regular grid-like arrangements, with heavy occlusion along shelf edges. The visual differences between products are often subtle — two coffee packages from the same brand may differ only in a "100g" vs. "200g" label, requiring near-OCR-level discrimination. Products always appear in a canonical upright orientation (unlike general object detection where objects can appear at any angle), which creates opportunities for domain-specific augmentation choices. The training set of 248 images is tiny by modern standards, making overfitting a constant threat.

Additionally, the competition provided studio-quality reference images of each product from multiple angles (front, back, sides, top, bottom). Several teams attempted to leverage these for gallery-based classification or synthetic data generation, with mixed results discussed in Section 8.

3 The Generalization Problem

The central challenge was not building a good detector — it was building one that generalizes. With 248 training images across 356 categories, every design decision carried the risk of overfitting. We identified three levels of generalization failure during the competition.

First, the **training-to-validation gap**. A YOLOv8l model with seed 77 achieved val mAP 0.804 but scored only 0.770 on the public test, while a YOLOv8x with lower val mAP scored 0.780. With roughly 37 validation images (15% of 248), local metrics are simply too noisy to be reliable.

Second, the **local-to-public gap**. Models trained with `cls=2.0` (double classification loss weight) scored +0.015 higher on training-set evaluation but showed zero improvement on the

public test. Classification-specialized models memorize training distributions rather than learning generalizable features.

Third, and most importantly, the **public-to-private gap**. The final private test set differed from the public leaderboard. Our submission moved from 4th on public to 1st on private, while competitors who had optimized for the public leaderboard dropped. This confirmed our generalization-first strategy.

Our strategy was informed by studying winning solutions from related Kaggle competitions, particularly Global Wheat Detection [David et al.(2020)] (domain shift in test set, WBF ensembles), Great Barrier Reef [GBR 2021] (dense detection with tiled training and WBF), VinBigData [VinBigData 2021] (low data, multi-architecture ensembles), and Sartorius [Sartorius 2021] (only 606 images, external pre-training critical). The recurring pattern across winners was clear: ensembles with WBF, architecture diversity, and aggressive data utilization.

These observations produced three design principles that guided all subsequent decisions: train on the full dataset without a validation split; maximize architectural diversity so that models fail independently; and reject any technique that shows strong local improvement but no test-set gain.

4 Method

4.1 Architecture Diversity

The core of our submission is a heterogeneous ensemble of three architectures, each contributing a different "view" of the data.

Table 1: The three models in the final ensemble.

Model	Architecture	Training Strategy	Role
fulldata_x	YOLOv8x (68.2M params)	Standard augmentation	Strong baseline
nf_11x_s42	YOLO11x [Ultralytics(2024)] (56.9M params)	No vertical flip	Different backbone
v8l_highaug	YOLOv8l (43.7M params)	Heavy augmentation	Regularized complen

The models differ along three orthogonal axes. Architecturally, YOLOv8x and YOLO11x have different backbone structures and feature pyramid designs, producing genuinely different error patterns. In terms of augmentation, the noflip model learns stronger vertical-orientation features while the heavy-augmentation model (mixup=0.3, copy-paste=0.3, rotation up to 10 degrees) learns more robust, regularized features. In terms of capacity, the range from 43.7M to 68.2M parameters gives different bias-variance tradeoffs.

This three-axis diversity means the models fail on different products, and WBF can correct individual errors through fusion. Homogeneous ensembles (3×YOLOv8x with different seeds) share failure modes and benefit less from fusion.

4.2 Training

All models were trained on the full 248-image training set without holding out a validation split. We found early on that withholding even 15% of data significantly hurts performance on this small dataset, and that the resulting validation metrics do not predict test performance.

Training used 150 epochs with cosine learning rate scheduling and AdamW optimization at 640×640 px input resolution. Mosaic augmentation was enabled with close-mosaic at epoch 130. All three models used seed 42 — diversity comes from architecture and augmentation choices, not random seed variation.

The noflip model (`flipud=0.0`) deserves special mention. Since grocery products are never upside-down on shelves, vertical flip augmentation creates unrealistic training examples. Dis-

abling it improved local classification scores by +0.008, and when combined with the YOLO11x architecture, contributed to the ensemble's generalization.

4.3 FP16 ONNX Export

The sandbox environment's PyTorch 2.6.0 breaks `.pt` model loading from Ultralytics due to a change in `torch.load()` defaults. We exported all models to ONNX with FP16 quantization, which also halved model size with negligible accuracy loss (<0.001 mAP). The three models total approximately 326 MB (131 + 110 + 85 MB), fitting within the 420 MB submission limit.

4.4 Multi-Scale Inference with TTA

Products on grocery shelves span a wide range of scales, from large cereal boxes to small tea packets. We run each model at both 640×640 px and 800×800 px, with horizontal-flip test-time augmentation at each scale. This produces $3 \times 2 \times 2 = 12$ inference passes per image. We tested 960 px as a third scale but found diminishing returns (−0.001 on test).

4.5 Weighted Boxes Fusion

Rather than Non-Maximum Suppression, we use Weighted Boxes Fusion [Solovyev et al.(2021)] to merge the 12 prediction sets. WBF produces more accurate bounding box coordinates by weighted averaging of matched detections rather than discarding overlapping boxes.

The `absent_model_aware_avg` confidence type is essential for our diverse ensemble. It averages scores only across models that actually produced a matching detection (N_{present}), rather than dividing by the total number of models M. This prevents the smaller YOLOv8l from dragging down confidence scores when it misses detections that the larger models catch. We use iou_thr = 0.6 and skip_box_thr = 0.01.

4.6 Mathematical Formulation

For completeness, we describe the key mathematical operations in the pipeline.

Letterbox preprocessing. Given an input image of size $(W_{\text{orig}}, H_{\text{orig}})$ and target size S, the letterbox transform computes a uniform scale factor $r = \min(S/W_{\text{orig}},\ S/H_{\text{orig}})$, resizes the image to $(W_{\text{new}}, H_{\text{new}}) = (\lfloor W_{\text{orig}} \cdot r \rfloor, \lfloor H_{\text{orig}} \cdot r \rfloor)$, and centers it on an $S \times S$ canvas with padding (p_x, p_y). Predicted coordinates are mapped back via $x_{\text{orig}} = (x_{\text{pred}} - p_x)/r$.

Weighted Boxes Fusion. Given M models producing box sets $\{\mathcal{B}_1, \ldots, \mathcal{B}_M\}$, WBF clusters overlapping boxes with IoU $> \tau_{\text{wbf}}$ and computes the fused box as a confidence-weighted average:

$$b_{\text{fused}} = \frac{\sum_{k=1}^{|C|} s_k \cdot b_k}{\sum_{k=1}^{|C|} s_k} \tag{2}$$

The absent-model-aware fused score is:

$$s_{\text{fused}} = \frac{\sum_{k=1}^{|C|} s_k}{N_{\text{present}}} \tag{3}$$

where $N_{\text{present}} \leq M$ counts only models that contributed a box to cluster C. Compare with the standard average $s_{\text{avg}} = \sum s_k / M$, which unfairly penalizes detections that only a subset of models find.

Mean Average Precision. For each class c, the Average Precision is the area under the precision-recall curve: $\mathrm{AP}_c = \int_0^1 p_c(r)\,dr$. A detection is a true positive if IoU ≥ 0.5 with a ground-truth box of the same class. The final metric is $\mathrm{mAP@0.5} = \frac{1}{|\mathcal{C}|}\sum_{c\in\mathcal{C}} \mathrm{AP}_c$, weighted into the competition score via Equation 1.

5 Technique Radar

Figure 2 provides a visual overview of all techniques explored during the competition. The path to our best result runs through architecture diversity and multi-scale inference, while post-processing and specialization directions consistently showed diminishing or negative returns.

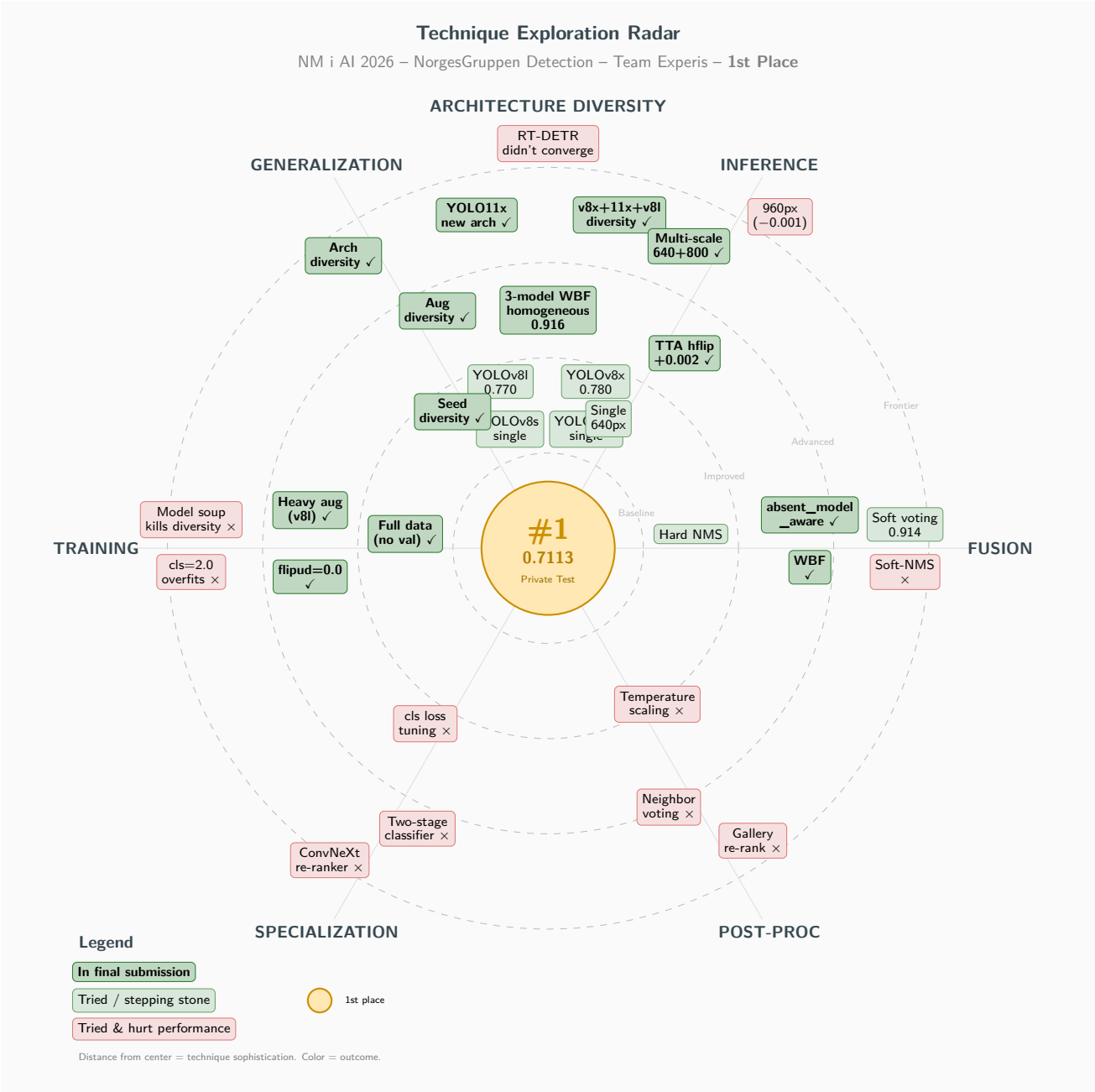

Figure 2: Technique exploration radar showing all approaches tried during the competition. The path to our best result runs through architecture diversity (North) and multi-scale inference (ENE). Post-processing and specialization (red nodes) consistently failed to generalize.

6 Experiments and Results

6.1 Submission History

Table 2 shows all competition submissions. The progression tells the story of our approach: the massive jump from single model to ensemble, followed by incremental gains from inference improvements and diversity.

Table 2: Competition submissions with public test scores. The final submission achieved 0.9226 on public and 0.7113 on private test.

#	Submission	Public Test	Δ	Key Change
1	`yolov8x_onnx_clean`	0.7802	—	Single model baseline
2	`yolov8l_640_s77`	0.7700	−0.010	Architecture variant
3	`onnx_ensemble_ms_tuned`	0.9091	+0.129	WBF ensemble (x+m+s)
4	`softvote_3x_fp16`	0.9142	+0.005	Soft class voting
5	`wbf_3x_fp16_tta`	0.9160	+0.002	WBF + TTA hflip
6	`wbf_2x1l_multiscale_tta`	0.9210	+0.005	Multi-scale + arch diversity
7	`candidate11_noflip_l`	0.9215	+0.001	Training diversity
8	`sub1_triple_arch`	0.9190	−0.003	YOLO11x first attempt
9	`sub2_yolo11x_swap`	**0.9226**	+0.001	**Architecture diversity (winner)**

6.2 Ablation

Table 3 isolates the contribution of each technique. Ensembling provides the single largest gain, with multi-scale and architecture diversity contributing the final margin.

Table 3: Ablation study on the public test set.

Configuration	Public Test	Δ
Single YOLOv8x (640px)	0.7802	—
+ WBF 3-model ensemble (same arch)	0.9091	+0.1289
+ TTA horizontal flip	0.9160	+0.0069
+ Multi-scale (640+800px)	0.9210	+0.0050
+ Architecture diversity (YOLO11x + YOLOv8l)	0.9226	+0.0016

6.3 What Did Not Generalize

Table 4 documents approaches that improved local metrics but failed on test. The most dangerous techniques were those with large local improvements — they build false confidence and waste limited submission budget.

Table 4: Techniques that failed to generalize.

Technique	Local Δ	Test Δ	Why it failed
cls=2.0 loss weight	+0.015	+0.000	Memorizes training classes
Two-stage classifier	+0.007	−0.011	96.9% train acc, overfits to crops
Gallery re-ranking	+0.000	−0.003	Can't distinguish fine-grained variants
Model soup (weight avg)	−0.015	N/A	Destroys diversity for ensembles
Neighbor class voting	−0.004	N/A	Flips correct predictions
960px third scale	N/A	−0.001	Diminishing returns

7 Tooling and Infrastructure

7.1 Training Infrastructure

All model training ran on Azure Machine Learning, using up to 8 nodes of Standard_NC4as_T4_v3 (NVIDIA T4, 16 GB VRAM each), generously provided by Experis. Over 69 hours we executed 150+ training runs spanning 7 architectures (YOLOv8x/l/m/s, YOLO11x, RT-DETR, YOLO26x), 14 random seeds, and numerous augmentation configurations. Job definitions were managed as YAML files, enabling rapid experiment dispatch.

In hindsight, we should also have leveraged the free GCP GPU credits provided to all competition teams by the organizers. We used GCP for the other two tasks but not for this one, which limited our parallel training throughput during the critical final hours.

7.2 Analytics Dashboard

We built a Streamlit-based analytics dashboard that provided real-time visibility into our experiment results. The dashboard tracked all training runs from `experiments.csv`, displayed detection versus classification mAP breakdowns from local evaluations, and most importantly identified which product categories each model struggled with.

This per-category failure analysis directly informed our architecture diversity strategy. By comparing confusion matrices across models, we could see that YOLOv8x and YOLO11x failed on *different* product pairs — confirming that combining them would produce complementary predictions rather than correlated errors.

7.3 Prediction Caching

To enable rapid post-processing sweeps, we cached each model's raw detections (boxes, scores, full 356-dim class probability vectors) on all 248 training images. This allowed testing all $\binom{6}{3} = 20$ three-model combinations from 6 candidate models in under 60 seconds, and sweeping 21 WBF parameter configurations in under 30 seconds — compared to 60+ minutes for a single full inference run.

7.4 Submission Validation

After an early `import sys` statement in a submission caused an automatic sandbox ban (the security scanner immediately blocks any submission containing prohibited imports), we built a comprehensive validation pipeline. A dedicated `validate_submission.py` script checks all sandbox rules: 23 blocked import modules, blocked function calls, file count and size limits, the 3-weight-file maximum, path traversal, binary executables, and output format compliance. A Pytest suite runs 10 automated tests per ZIP before every build, and a Docker-based local sandbox replica provides end-to-end validation. This pipeline prevented at least two additional invalid submissions.

8 Discussion

Post-competition discussion in the official Slack channel revealed that many teams explored similar directions, arriving at converging conclusions. This section summarizes shared findings and open questions from the community.

8.1 Synthetic Data

Synthetic data generation was one of the most popular strategies, with at least six teams reporting results. André Mikalsen (0.920) used Nano Banana 2 to generate product image variations

in shelf-realistic contexts (different angles, shadows, lying down). Andreas Hatlem (0.9220) generated approximately 1,400 synthetic images by compositing reference photos onto shelf backgrounds. Markus Hei physically visited stores to photograph underrepresented products and used Gemini with SAM3 for silver labeling. Dag Koding from DKIT experimented with synthetic data and even considered 3D models of product packaging. Ludvig Øvrevik produced 3,000 synthetic images using Nano Banana. Jørgen Rørvik generated 50 images per product class using Nano Banana with cropped references.

The consistent finding across all teams was that synthetic data did not improve scores. We did not attempt synthetic data ourselves, instead investing time in architecture diversity. In retrospect, we believe the domain gap between studio product photos and real shelf crops is the primary obstacle — compositing techniques cannot replicate the complex interactions of lighting, occlusion, price tags, and shelf geometry that characterize real retail images. Additionally, synthetic data risks reducing ensemble diversity if all models train on the same generated artifacts.

8.2 Converging Findings

Several findings were independently confirmed by multiple teams. Training on all 248 images (without a validation split) was reported as a significant improvement by Hatlem (+0.004) and matches our experience. Mikalsen worked around the lack of validation data by generating synthetic validation images with Gemini 3.1 Flash for classification labels. Multi-model WBF ensembles were the dominant strategy among top teams. Hatlem independently discovered the same diversity principle we relied on: his weakest individual model (K-fold 2, 0.749 mAP) consistently gave the best ensemble results, and swapping it for stronger models always hurt the ensemble score. Walgermo from Matriks (0.9208) also used a 3-model ensemble with random seed diversity. Post-processing tuning (WBF parameters, Soft-NMS, temperature scaling) was reported as unhelpful by both Hatlem and ourselves.

Eirik Bækkelund took a different approach with DINOv2 ViT-B/14 and an ArcFace head for classification, achieving roughly 94% accuracy on reference embeddings. His main issue was coffee brand confusion (Ali Kaffe alone accounted for 20% of errors), and he noted that lightweight OCR models could potentially resolve these confusions within the 420 MB file size budget.

8.3 Open Questions

The unknown product problem. Pål Bentsen raised an interesting question about the task specification: should products present in the image but not belonging to any of the 356 annotated categories be detected and classified as "unknown," or ignored entirely? As shown in Figure 3, the Natreen sweetener products on the shelf have no ground truth annotations despite being clearly visible. In a real-world retail system, detecting all products would be essential for inventory management. The competition's handling of this edge case was ambiguous, and the answer directly affects model design — detecting products that the evaluator ignores generates false positives that hurt the score.

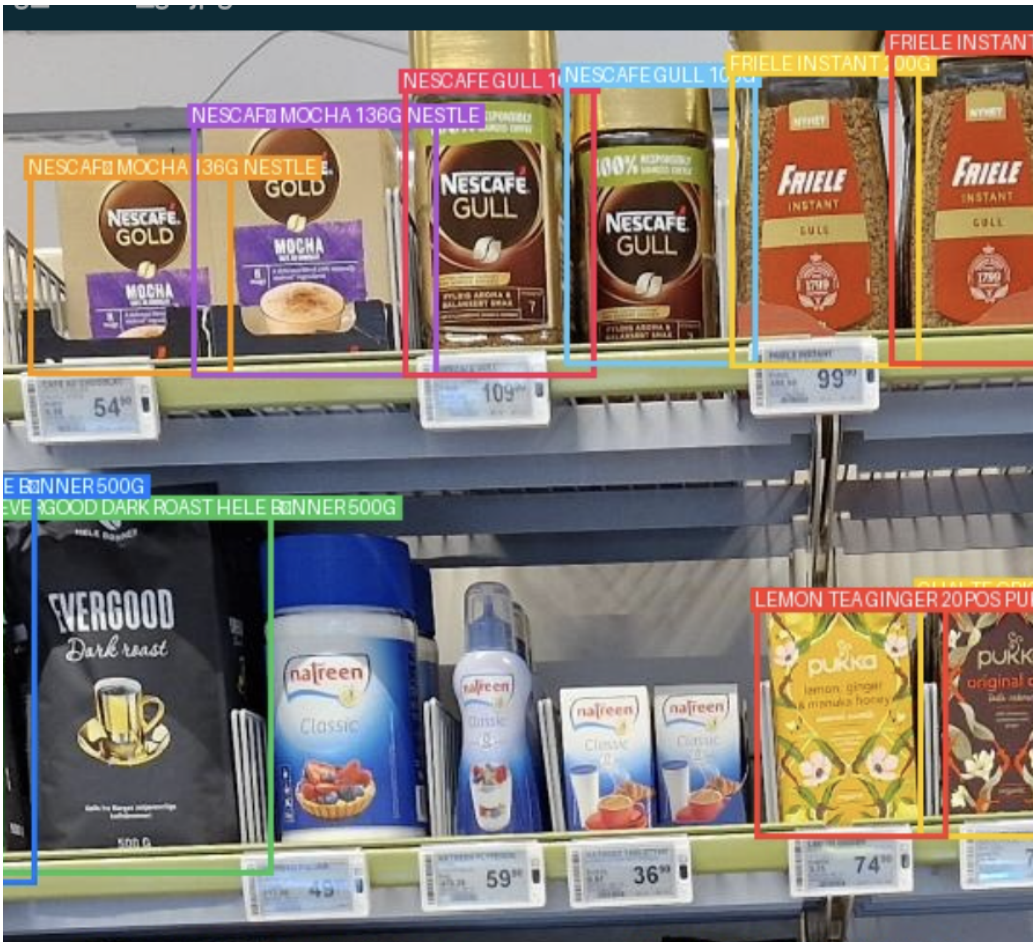

Figure 3: The unknown product problem, raised by Pål Bentsen. The Natreen products on the lower shelf have no ground truth annotations despite being clearly visible. In a real-world system these should be detected; in the competition, detecting them may hurt the score.

Annotation quality. Bentsen also observed cases where the ground truth contained errors — correct model predictions that lacked corresponding annotations. They manually corrected a selection and retrained, an approach that highlights the tension between trusting provided labels and investing time in data quality.

9 Lessons Learned

Generalization beats optimization. Our most important lesson is that optimizing for a known test set is risky when the final evaluation uses a different distribution. Our public leaderboard score (0.9226, 4th place) did not predict our private test result (0.7113), and the gap between public and private rankings was significant across teams. Designing for robustness rather than maximizing a single metric proved more reliable.

Diversity over specialization. In our experiments, every attempt at specialization (classification loss tuning, two-stage classifiers, gallery re-ranking, model soups) failed to generalize. Every form of diversity (architecture, augmentation, scale) helped. On small datasets, the lesson is to invest in making *different types of predictions* rather than *better predictions of the same type*.

Reject locally-good changes. The hardest discipline was rejecting techniques that showed clear local improvements. Our `cls=2.0` model scored +0.015 higher locally — a massive improvement by any measure. But it showed zero test improvement. Learning to distinguish

genuine gains from overfitting artifacts, without a reliable validation set, was one of the hardest challenges of the competition.

The domain matters. Grocery shelf recognition has strong structural priors (upright products, grid layouts, canonical lighting) that reward domain-aware choices like disabling vertical flip augmentation. But it also has adversarial properties (near-identical products differing only in small text) that defeat standard feature extractors. Understanding both aspects shaped our approach.

10 Conclusion

We presented our solution to the NM i AI 2026 NorgesGruppen product detection task, achieving 0.7113 mAP on the private test set. Our approach combined three YOLO architectures (YOLOv8x, YOLO11x, YOLOv8l) with diverse augmentation strategies, multi-scale inference, and Weighted Boxes Fusion — without any synthetic data, external datasets, or post-processing tricks. The primary lesson for practitioners: when the test distribution is uncertain, bet on diversity, not accuracy.

Acknowledgments

We thank **Experis** for providing the Azure ML compute resources that enabled our 150+ training runs — without dedicated GPU access, the scale of experimentation described here would not have been possible. We also thank the NM i AI organizers at **Astar Technologies** for running an excellent competition, **NorgesGruppen Data** for providing a genuinely challenging and well-crafted task, and all the participants whose creative approaches (documented in Section 8) made the post-competition discussion as educational as the competition itself. We note that free GCP GPU credits were provided to all teams by the organizers; we used these for Tasks 1 and 2 but not for this task, which in hindsight was a missed opportunity for additional training throughput.

References

[Jocher et al.(2023)] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLO, 2023. https://github.com/ultralytics/ultralytics.

[Solovyev et al.(2021)] R. Solovyev, W. Wang, and T. Gabruseva. Weighted boxes fusion: Ensembling boxes from different object detection models. *Image and Vision Computing*, 107:104117, 2021.

[Lin et al.(2014)] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *ECCV*, 2014.

[Goldman et al.(2019)] E. Goldman, R. Herzig, A. Eisenschtat, J. Goldberger, and T. Hassner. Precise detection in densely packed scenes. In *CVPR*, 2019. Dataset: https://docs.ultralytics.com/datasets/detect/sku-110k/.

[David et al.(2020)] E. David, S. Madec, P. Sadeghi-Tehran, et al. Global wheat head detection (GWHD) dataset. *Plant Phenomics*, 2020. Kaggle: https://www.kaggle.com/competitions/global-wheat-detection.

[GBR 2021] Kaggle. TensorFlow — Great Barrier Reef. https://www.kaggle.com/competitions/tensorflow-great-barrier-reef, 2021. 1st place: 6-model YOLOv5 ensemble with tiled training and WBF.

[VinBigData 2021] Kaggle. VinBigData Chest X-ray Abnormalities Detection. https://www.kaggle.com/competitions/vinbigdata-chest-xray-abnormalities-detection, 2021. 1st place: YOLOv5 + EfficientDet ensemble with WBF, 15K images, 14 classes.

[Sartorius 2021] Kaggle. Sartorius Cell Instance Segmentation. https://www.kaggle.com/competitions/sartorius-cell-instance-segmentation, 2021. 606 images only; winner used external pre-training on LIVECell.

[Ultralytics(2024)] Ultralytics. YOLO11: Next-generation real-time object detection, 2024. https://docs.ultralytics.com/models/yolo11/.